

Live coding 101

Penny Anastasopoulou, Pablo Tablas

What is live coding?



Artistic Practice

Giving instructions to computers in the form of algorithms.



Real Time

Combines code with the creation of music (or visuals) in real time.



Origins

The term was first used in 2003!



Creation

Improvisation understood as a conversation between human and machine.



Programs are instruments that can change themselves.

Philosophy and approach



We are not DJs!

(community's inside joke, so alex, no, it is not a roasting to DJs)

Music creators, improvisers, composers, not curators of prerecorded music.



Ethos, transparency, openness

- Projectors allow the audience to see the code being modified live.

"Obscurantism is dangerous. Show us your screens." [\(TOPLAP manifesto\)](#)

"Give us access to the performer's mind, to the whole human instrument."

- Focus on ideas and processes rather than software itself.

"Live coding is not about tools. Algorithms are thoughts. Chainsaws are tools..."

- Risk and unpredictability are embraced as part of the performance.

"We prefer no backup."

General tips



Embrace mistakes

Errors are part of the process. In live coding, "bugs" often lead to unexpected musical discoveries.



Simple rules

Complexity grows from simple rules. Layer basic patterns to create cute music-scapes.



Musical impact

Small edits can have big musical impact. One character change can shift the entire rhythm.

Strudel: live coding on the web

<https://strudel.cc/>



cycle: repeating structure (instead of bpm)



think in cycles: every change should relate to the loop you're already in



a loop that keeps evolving



pattern-based composition

Basics #1

Notes and sounds

Note-based or event/sound-based trigger/sequencing.

```
s("bd hh")
```

```
note("c3 eb3") or note("48 51")
```

< > Alternate

One choice per cycle (repeats when done).

```
s("<bd sd hh>") // bd on cycle 1, sd on cycle 2..
```

[] Subdivision

Fits multiple events/notes/hits into a single step.

```
s("bd [sd sd] hh") // the [sd sd] squeezes two hits in one  
step (if sd was a quarter, it's now two eighths)
```

[, ,] Simultaneous layers

Comma creates a new parallel layer.

Polyrhythms:

```
s("[bd bd bd, hh hh hh hh]") // 3 against 4
```

Chords:

```
note("<[c3, eb3, g3] [f3, a3, c4]>")
```

Basics #2: transformation

The dot (`.`) operator

-> The dot connects a pattern to a transformative action. You can **chain** transformations for pattern modifying, audio effects, signal modulation, sound selection, and more.

Pattern control

- **fast/slow**: Changes playback speed of the entire pattern (multiplied).
- **rev**: Reverses the pattern content.
- **jux**: Juxtaposes stereo with contrasting effects (accepts function of transformation).

Audio effects

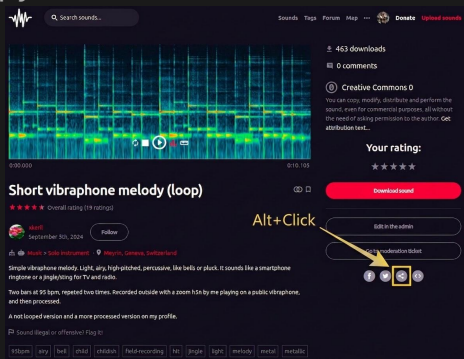
- **lpf/hpf**: Low/high-pass filter frequency (in Hz).
- **velocity**: Note gain/intensity (0-1).
- **room**: Reverb amount (0-1).
- **pan**: Stereo position (-1 to 1).
- **delay**: Level of the delay signal (0-1).

Example: `s("bd bd").lpf(500).room(0.5)`

More samples in Strudel: Freesound

↔ Freesound Integration from UI

1. Visit freesound.org
2. Find a sound: [e.g. #753346](#)
3. Copy sound link:



e.g. https://cdn.freesound.org/previews/753/753346_13504080-hq.mp3

4. Paste in strudel:

```
samples({ example: '<link>' })  
s("example")
```

🔧 Shabda Tool

[Native Strudel tool](#) for instant querying of Freesound samples.

Query syntax:

```
samples('shabda:laugh:4')  
# shabda:query_word:number_of_samples
```

Listen to your results:

```
s("laugh:0")    //first sample  
  
n("0 1 2 3").s('laugh').slow(2).clip(1)
```


Basics #3: sample-based thingies

Sample transformation

- **speed**: Sample playback rate (multiplied, negative reverses).
- **clip**: Plays sample for given duration, cuts if longer, gaps if shorter.
- **loop**: Loops the sample to fill the duration.
- **cut**: Cuts the sample when a new one in the same group triggers.
- **chop**: Cuts each sample into the given number of parts (granular synthesis thingies).

Patterns together

stack plays patterns in parallel, and you can apply effects to multiple patterns at the same time!

```
stack(  
  s("bd sd"),  
  s("hh hh hh hh")  
) .room(0.2)
```

Let's try to play and jam ourselves!

Go beyond the given examples and more info here:

<https://strudel.cc/learn/>